Mississippi Valley State University

CS 341 — Discrete Structures

Practice Set 43: Approved Solutions

Algorithms and Time Complexity

Covers Chartrand & Zhang, Sections 6.1 and 6.3

Status of this set

Ungraded practice. Solutions released with it. Show the count, not just the answer.

Part I: How to Practice This Material

Counting loop work

- A loop running 1 to n with one comparison inside: n comparisons.
- Independent nested loops: **multiply**.
- Sequential loops: **add**.
- An inner loop whose bound depends on the outer index: **sum** the inner counts.

That last case is where the Week 11 summation formulas earn their keep.

Three mistakes to expect

1. **Off by one on loop bounds.** "$i := 2$ to n" runs $n - 1$ times, not n.
2. **Multiplying when the inner bound varies.** That calls for a sum.
3. **Confusing worst case with average case.** Say which you are computing.

Part II: Practice Problems

Section A — Counting

1. Give the exact number of comparisons each structure performs on an input of size n.

(a) A single loop $i := 1$ to n with one comparison inside.

(b) Nested loops, outer $i := 1$ to n, inner $j := 1$ to n, one comparison inside.

(c) Outer $i := 1$ to n, inner $j := 1$ to i, one comparison inside.

(d) Outer $i := 1$ to $n - 1$, inner $j := i + 1$ to n, one comparison inside.

Approved solution

(a) n.
(b) $n \cdot n = n^2$ — independent loops multiply.
(c) $1 + 2 + \cdots + n = \dfrac{n(n+1)}{2}$. The inner bound depends on i, so the counts are summed, not multiplied.
(d) $(n-1) + (n-2) + \cdots + 1 = \dfrac{n(n-1)}{2}$. This structure compares every *pair* exactly once, which is why the count is $\binom{n}{2}$.

Parts (c) and (d) both use the Week 11 sum. And (d) is a Week 13 count in disguise: comparing all pairs of n items is $\binom{n}{2}$.

2. An algorithm finds both the largest and smallest number in a list of n numbers by comparing each new term first against the current maximum, and if that fails, against the current minimum.

(a) How many comparisons in the worst case?

(b) How many in the best case?

(c) Describe the input that produces each.

Approved solution

(a) $2(n-1)$**.** Each of the $n-1$ terms after the first may need both comparisons.
(b) $n-1$**.** If every term beats the current maximum, the second comparison is never reached.
(c) Worst case: a strictly decreasing list, so no term is ever a new maximum and every one falls through to the minimum test. Best case: a strictly increasing list.

This is the first algorithm in the set with a genuine best/worst gap. Algorithm 6.4 had none — it makes $n-1$ comparisons on every input, because the comparison happens whether or not it succeeds.

Section B — Worst, best, average

3. A linear search runs on a list of n distinct numbers.

(a) Best, worst, and average case when the target is present, each position equally likely.

(b) What is the worst case when the target is *absent*?

(c) For $n = 100$, give all four numbers.

Approved solution

(a) Best 1; worst n; average $\dfrac{1 + 2 + \cdots + n}{n} = \dfrac{n+1}{2}$.
(b) n — the whole list is scanned, same as the worst case when present.
(c) For $n = 100$: best 1, worst 100, average 50.5, absent 100.

The average is only half the worst case, which is a constant factor and therefore invisible to big-O. Both are $O(n)$. That is the point of the next lecture.

Section C — Writing

4. In four to six complete sentences, explain why counting comparisons is a better measure of an algorithm than timing it on a particular computer.

Approved solution

Model answer. The time a program takes depends on the machine, the language, the compiler, and what else the machine is doing, none of which is a property of the algorithm itself. Replacing the hardware with something twice as fast halves the running time for every input size, so it changes the measurement by a constant factor and tells you nothing new about how the work scales. Counting comparisons strips those factors away and leaves a function of the input size alone, which is what distinguishes one algorithm from another. That count can then be compared across algorithms and across machines, and it predicts what happens as the input grows — which is the question that actually matters.

Full credit requires the point that hardware changes the time by a constant factor and so does not change the comparison between algorithms.

Section D — Challenge (optional)

5. Show that finding both the maximum and the minimum can be done in about $\frac{3n}{2}$ comparisons rather than $2(n-1)$.

Approved solution

Process the list in **pairs**. For each pair, first compare the two elements against each other (1 comparison), then compare the larger against the running maximum and the smaller against the running minimum (2 more). That is 3 comparisons per pair, and there are $n/2$ pairs, giving about $\dfrac{3n}{2}$. Compare with $2(n-1) \approx 2n$ for the naive method — a saving of about a quarter.

Both are $O(n)$. The improvement is a constant factor, which big-O discards. That is not a flaw in big-O: a constant factor is worth having in practice and is still the right thing to ignore when asking which algorithm scales better.

Instructor notes

- **Problem 1(c) and 1(d)** both need a summation, not a product. If students multiply, they have not noticed the dependent bound.
- **Problem 2** is the first algorithm here with a real best/worst gap — worth contrasting with Algorithm 6.4, which has none.
- **Problem 5** previews the limits of big-O before big-O is even defined: a constant-factor win is real but invisible to the notation.
- Reserved for Homework 15: Section 6.1 even-numbered exercises.